

Remediación de Vulnerabilidades OWASP

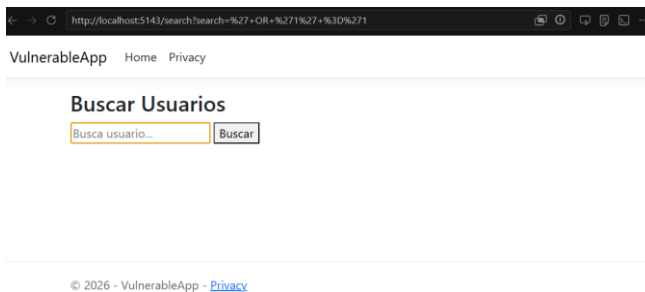
Antes y Después Vulnerabilidades

SQL Injection en la sección de búsqueda de usuarios.

Buscar Usuarios

Username	Email	Balance
admin	admin@test.com	\$1000.00
user1	user@test.com	\$500.00
user2	user2@test.com	\$750.00

Antes



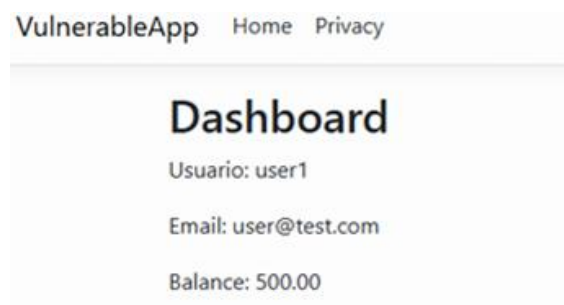
© 2026 - VulnerableApp - [Privacy](#)

Después

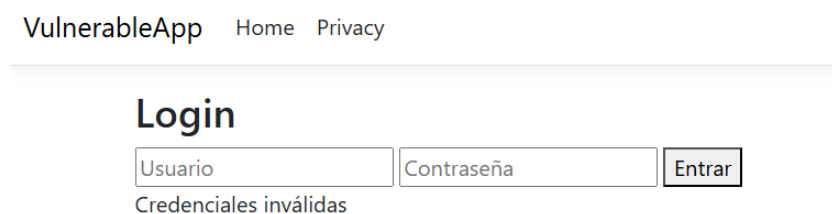
Código corregido

```
0 references
5 public IActionResult Index(string search)
6 {
7     if (string.IsNullOrEmpty(search))
8         return View(new List<User>());
9     var users = _db.Users
10        .Where(u => u.Username.Contains(search))
11        .ToList();
12     return View(users);
13 }
14 }
15 }
```

Antes y Después de Autenticación



Antes



Después

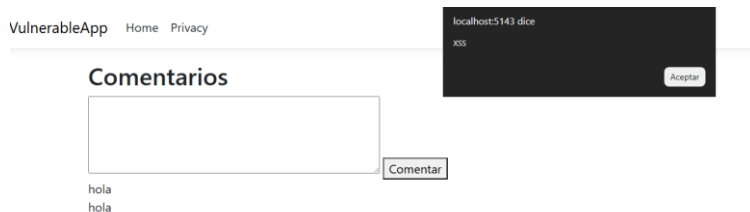
Código corregido

```
[HttpPost]
0 references
public IActionResult Login(string username, string password)
{
    var user = _db.Users.FirstOrDefault(u => u.Username == username);

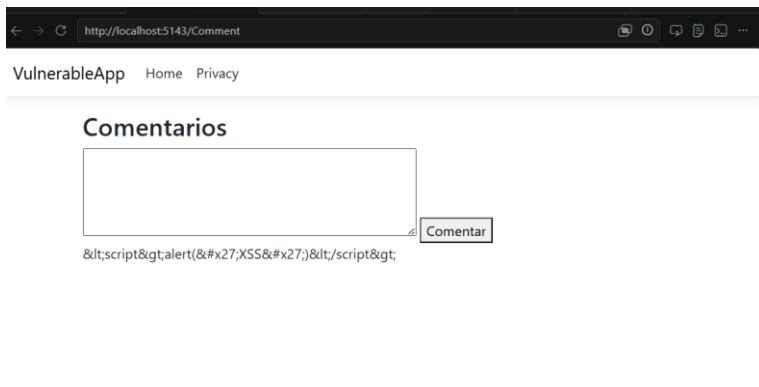
    if (user == null || !BCrypt.Net.BCrypt.Verify(password, user.PasswordHash))
    {
        ViewBag.Error = "Credenciales inválidas";
        return View();
    }

    HttpContext.Session.SetString("User", user.Username);
    HttpContext.Session.SetInt32("UserId", user.Id);
    return RedirectToAction("Dashboard");
}
```

Antes y Después XSS



Antes

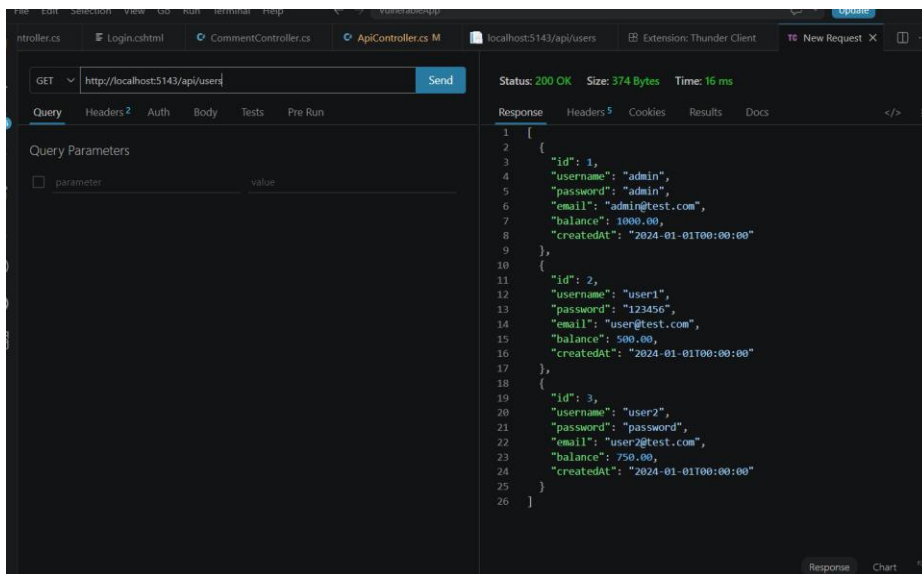


Después

Código corregido

```
</form>
<div id="comments">
  @foreach (var string comment in Model ?? new List<string>())
  {
    <div class="comment">@Html.Encode(comment)</div>
  }
</div>
```

Antes y Después IDOR



Antes

```

GET http://localhost:5143/api/users
Status: 401 Unauthorized Size: 165 Bytes Time: 34 ms
Response Headers 5 Cookies Results Docs
1 {
2   "type": "https://tools.ietf.org/html/rfc9110#section-15.5.2",
3   "title": "Unauthorized",
4   "status": 401,
5   "traceId": "00-2f3bc574012efc11baadaf03747ff69d-9dc0511c7d671f8a-00"
6 }

```

Después

Tabla Comparativa:

Vulnerabilidad	Archivo	Código Vulnerable	Código Corregido
Inyección SQL	Controllers/SearchController.cs	<pre>string query = "SELECT * FROM Users WHERE Username LIKE '%" + search + "%'"; var users = _db.Users.FromSql Raw(query);</pre>	<pre>var users = _db.Users.Where(u => u.Username.Contains(search)).ToList();</pre>
Autenticación Rota	Controllers/AuthController.cs / Data/AppDbContext.cs	<pre>string query = "SELECT * FROM Users WHERE Username = '" + username + "' AND Password = '" + password + "'";</pre>	<pre>var user = _db.Users.FirstOrDefault(u => u.Username == username); if (user == null</pre>
Cross-Site Scripting (XSS)	Views/Comment/Ind ex.cshtml	@Html.Raw(comment)	@Html.Encode(comment)
IDOR	Controllers/ApiController.cs	<pre>var user = _db.Users.Find(id); return Ok(new { ... user.Password });</pre>	<pre>if (id != currentUserId.Value) return Forbid(); return Ok(new { user.Id, user.Username, user.Email });</pre>